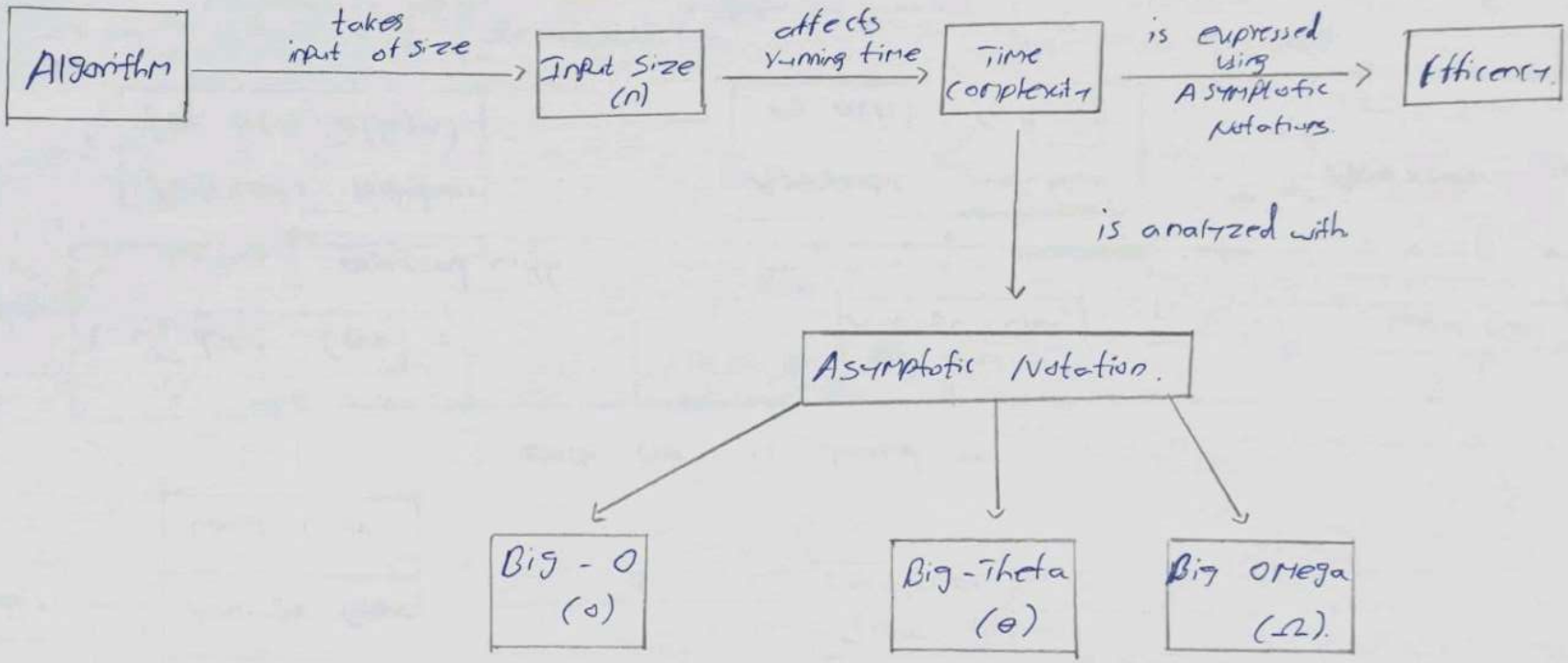
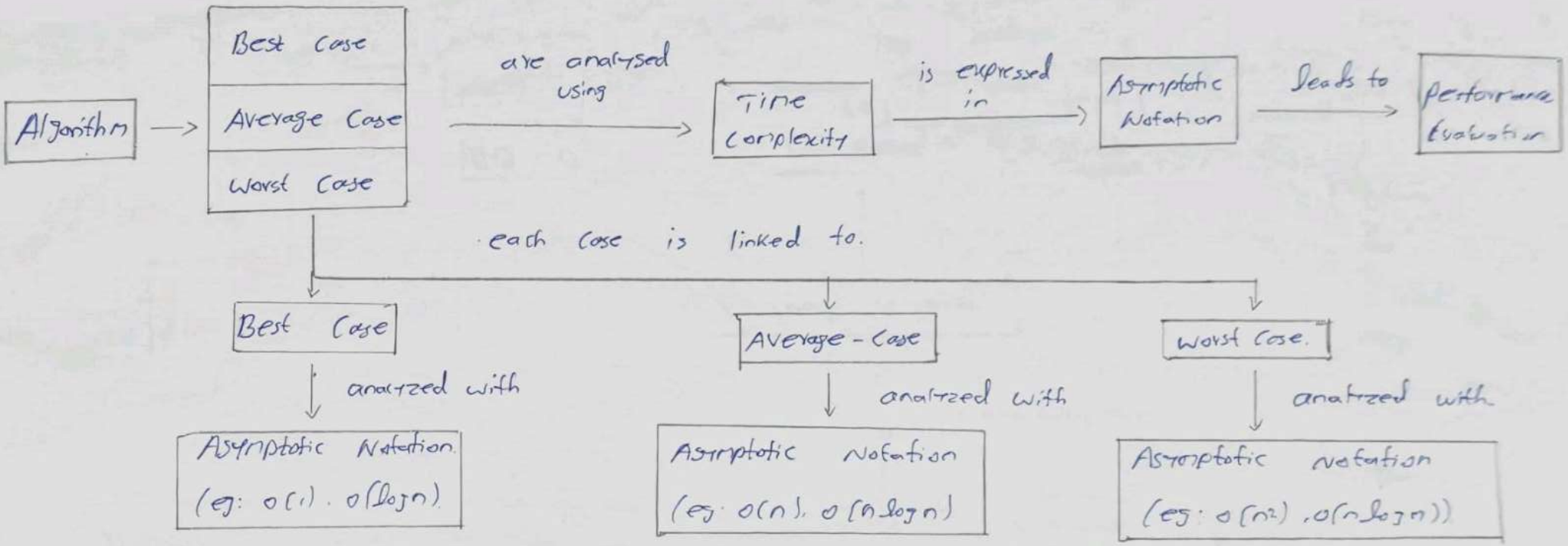
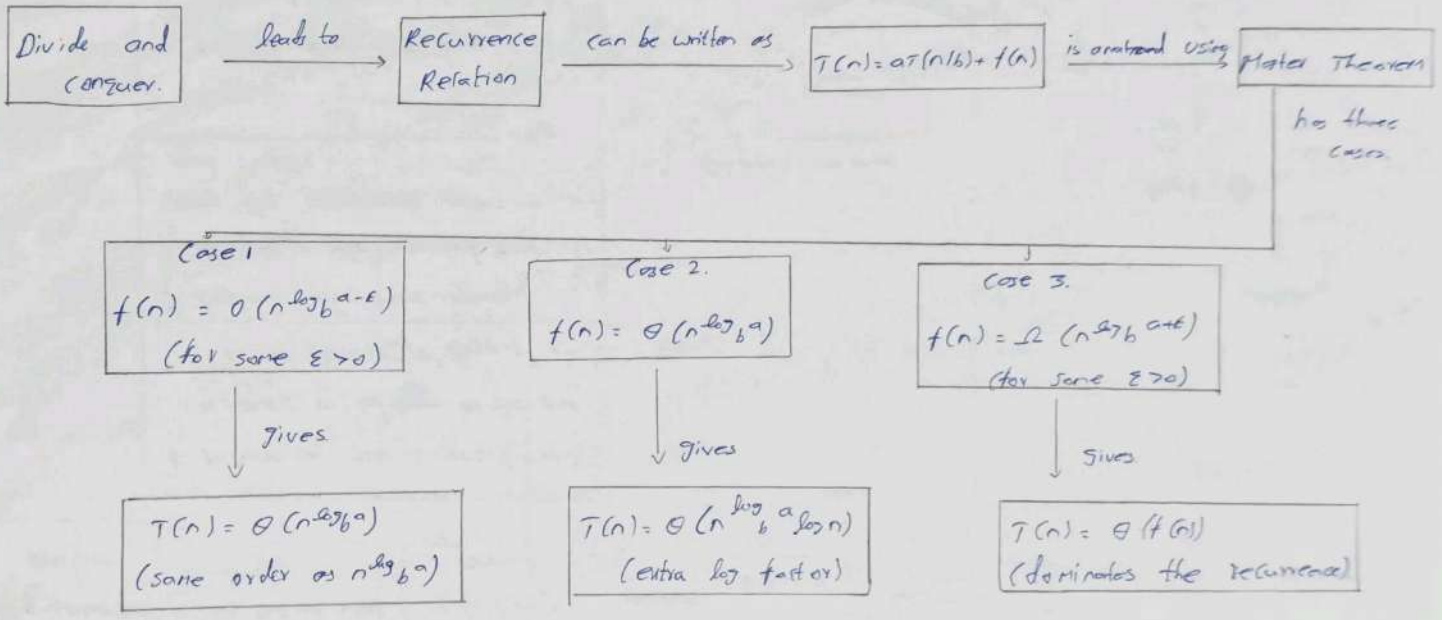
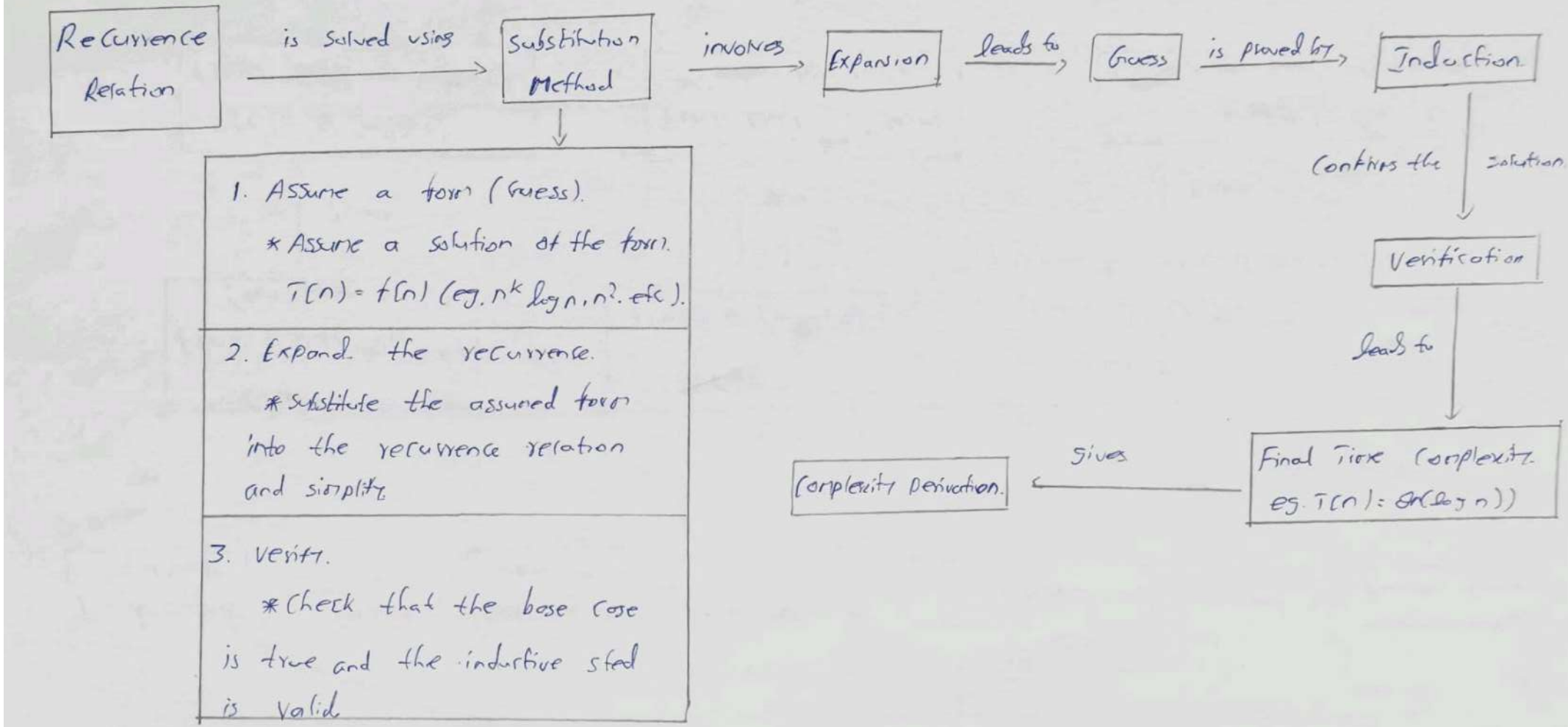


DAA - Col - AT 2

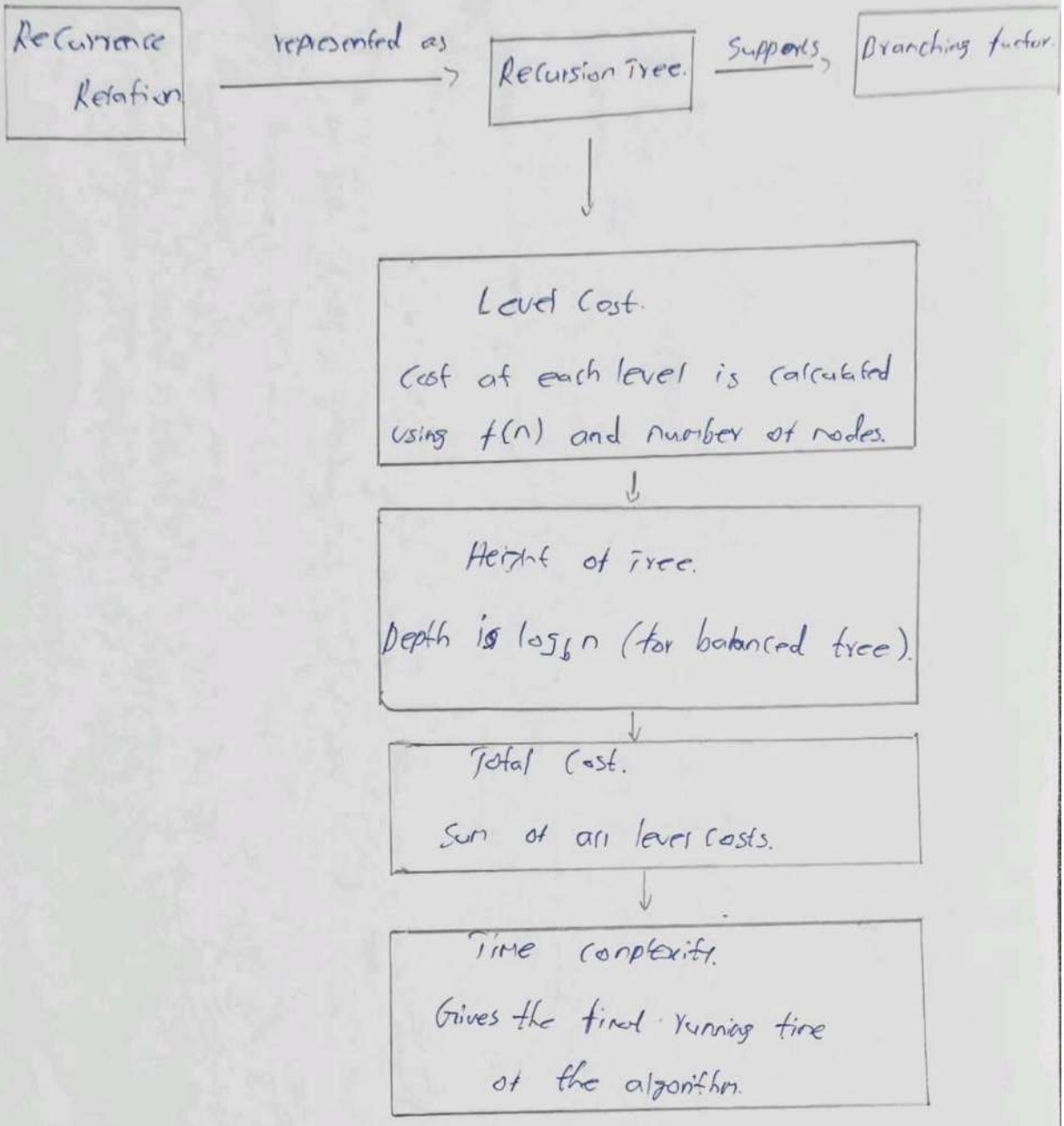
Thanigaivel S.
1715650006.

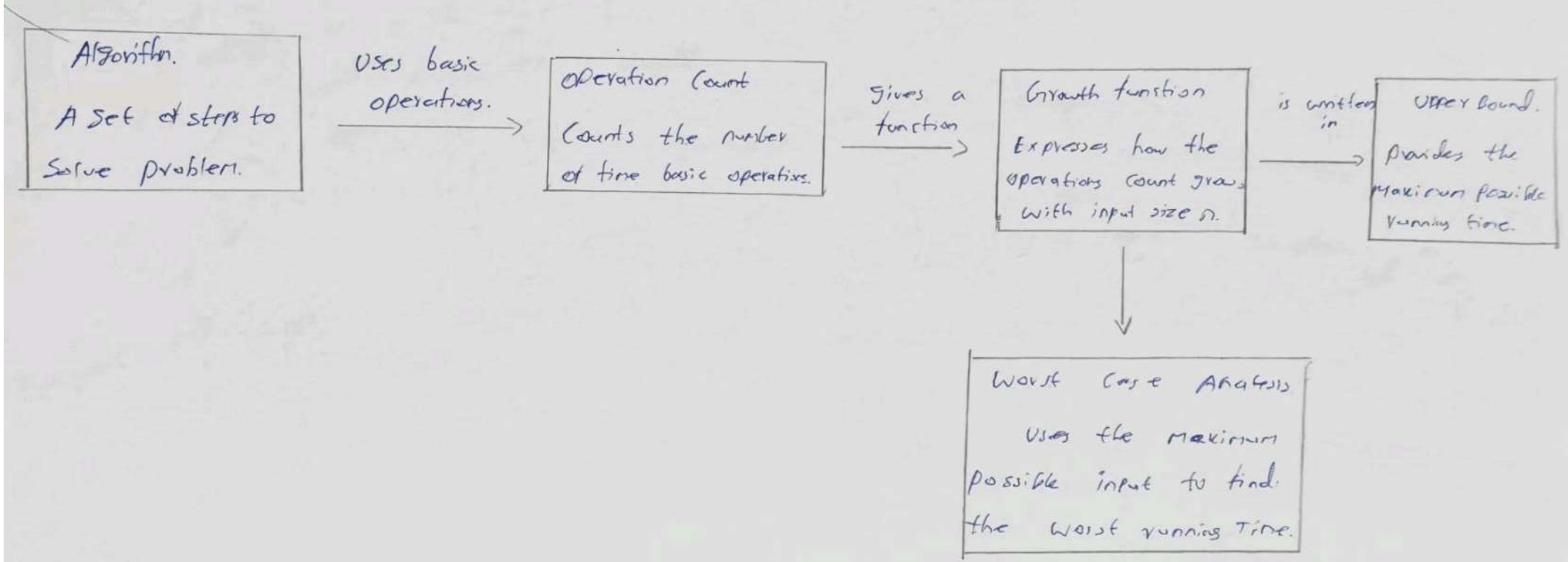

Algorithm
Best Case
Average Case
Worst Case
are analysed using
Time complexity
is expressed in
Asymptotic Notation
leads to
performance evaluation
each case is linked to.
Best Case
analyzed with
Asymptotic Notation
(eg: O(1), O(log n))
Average-Case
analyzed with
Asymptotic Notation
(eg: O(n), O(n log n))
Worst Case.
analyzed with
Asymptotic notation
(eg: O(n²), O(n log n))


Divide and conquer.
leads to
Recurrence Relation
can be written as
$T(n) = aT(n/b) + f(n)$
is evaluated using
Master Theorem
has three cases
Case 1
$f(n) = O(n^{\log_b a - \varepsilon})$
(for some $\varepsilon > 0$)
gives
$T(n) = \Theta(n^{\log_b a})$
(same order as $n^{\log_b a}$)
Case 2.
$f(n) = \Theta(n^{\log_b a})$
gives
$T(n) = \Theta(n^{\log_b a} \log n)$
(extra log factor)
Case 3.
$f(n) = \Omega(n^{\log_b a + \varepsilon})$
(for some $\varepsilon > 0$)
gives
$T(n) = \Theta(f(n))$
(dominates the recurrence)


Recurrence Relation
is solved using
Substitution Method
involves
Expansion
leads to
Guess
is proved by
Induction
confirms the solution
Verification
leads to
Final Time Complexity
eg. T(n) = Θ(log n))
gives
Complexity Derivation.
1. Assume a form (Guess).
* Assume a solution of the form.
T(n) = f(n) (eg. n^k log n, n^2. etc).
2. Expand the recurrence.
* Substitute the assumed form into the recurrence relation and simplify
3. Verify.
* Check that the base case is true and the inductive step is valid

5.


Recurrence Relation
represented as
Recursion Tree.
Supports
Branching factor.
Level Cost.
Cost of each level is calculated using f(n) and number of nodes.
Height of tree.
Depth is $\log_b n$ (for balanced tree).
Total Cost.
Sum of all level costs.
Time complexity.
Gives the final running time of the algorithm.


Algorithm.
A set of steps to solve problem.
Uses basic operations.
Operation Count
Counts the number of time basic operations.
gives a function
Growth function
Expresses how the operations count grow with input size n.
is written in
Upper bound.
Provides the maximum possible running time.
Worst Case Analysis
Uses the maximum possible input to find the worst running Time.

Algorithm.

* Solve a problem using a step by step process.
* Its solving time can often be expressed as a recurrence relation.

depends on.

Input size.

* Represents the size of the problem.
* used in the recurrence relation to show how the problem reduces.

leads to

Recurrence Relation.

* Expresses $T(n)$ in terms of smaller inputs.
* General form: $T(n) = aT(n/b) + f(n)$.

solved using

Methods to solve.

* Substitution method: assume and prove.
* Recursion Tree method: visualize the work done.

one of the methods

Master theorem.

* used for recurrences of the form: $T(n) = aT(n/b) + f(n)$
* Gives the asymptotic solution based on the values of a, b.

Linear Search.

*Checks elements one by one from the beginning.

*works on unsorted or sorted data.

*simple to implement.

Search Algorithm

is a

used to find a target element. in a list or array.

includes

Binary Search

* Compares with middle element.

*works only on sorted data.

Compared based on.

Key difference

*Data requirement:

* Time complexity.

*

Working steps

Linear search

*start from first element

* Compare with target.

binary search

*find middle element

* Compare with target.

Definition.

* Consists of vertices and edges.
* Used to model real world. networks

Graphs.

* A data structure used to represent relationships between objects.

Types.

* undirected Graph.
* Directed Graph
* weighted Graph.

Graph representation.

* Adjacency matrix
* Adjacency matrix
* Incidence matrix.

Graph Traversals

* Breadth first search
* Depth first search

Linear search.

Check each element one by one.
from the beginning until the target.
is found or the list ends.

Key Points - Linear Search

* Works on unsorted or sorted data.
* Checks one by one.
* Time complexity
* simple to understand and implement.

Binary search.

Compares the middle element.
and continues in the half where
the target may be

Key points - Binary Search.

* works only on sorted data.
* uses divide and conquer approach
* Time complexity: O (log n)
* faster for target data.